Computational Mathematics Lab

UNIT - I: Eigen values and Eigenvectors: **6P**

Programs:

1. Finding real and complex Eigen values.
2. Finding Eigen vectors.

UNIT-II: Solution of Algebraic and Transcendental Equations **6P**

Bisection method, Newton Raphson Method

Programs:

3. Root of a given equation using Bisection method.
4. Root of a given equation Newton Raphson Method.

UNIT-III: Linear system of equations: **6P**

Jacobi's iteration method and Gauss-Seidal iteration method

Programs:

5. Solution of given system of linear equations using Jacobi's method
6. Solution of given system of linear equations using Gauss-Seidal method

UNIT-IV: First-Order ODEs **8P**

Exact and non-exact equations, Applications: exponential growth/decay, Newton's law of cooling.

Programs:

7. Solving exact and non-exact equations
8. Solving exponential growth/decay and Newton's law of cooling problems

UNIT-V: Higher order linear differential equations with constant coefficients **6P**

Programs:

9. Solving homogeneous ODEs
10. Solving non-homogeneous ODEs

TEXT BOOKS:

1. MATLAB and its Applications in Engineering, Rajkumar Basal, Ashok Kumar Geo, Manoj Kumar Sharma, Pearson publication.
2. Kenneth A. Lambert, The fundamentals of Python: First Programs, 2011, Cengage Learnings.
3. Think Python First Edition, by Allen B. Downey, Orielly publishing.
4. Introduction to Python Programming, William Mitchell, Povel Solin, Martin Novak et al., NC Lab Public Computing, 2012.
5. Introduction to Python Programming, ©Jacob Fredslund, 2007.

REFERENCE BOOKS:

1. An Introduction to Python, John C. Lusth, The University of Alabama, 2011.
2. Introduction to Python, ©Dave Kuhlman, 2008.

Introduction to MATLAB

MATLAB (short for **MAT**rix **LAB**oratory) is a high-level programming language and interactive computing environment developed by MathWorks. It is widely used for numerical computation, data analysis, visualization, simulation, and algorithm development.

Features of MATLAB

- Easy-to-learn programming language.
- Powerful matrix and array operations.
- Built-in mathematical and engineering functions.
- High-quality 2D and 3D plotting.
- Tools for data analysis and visualization.
- Supports simulation and modeling (e.g., Simulink).
- Large collection of specialized toolboxes for fields such as signal processing, image processing, machine learning, and control systems.

Applications of MATLAB

MATLAB is commonly used in:

- Engineering (electrical, mechanical, civil, etc.)
- Scientific research
- Data analysis and statistics
- Artificial intelligence and machine learning
- Image and signal processing
- Robotics and automation
- Financial modeling

Basic MATLAB Window

The MATLAB desktop includes:

1. **Command Window** – Execute commands directly.
2. **Workspace** – View variables stored in memory.
3. **Current Folder** – Browse project files.
4. **Command History** – Review previously executed commands.
5. **Editor** – Write and save scripts and functions.

Advantages of MATLAB

- Simple and user-friendly syntax.
- Fast development for mathematical problems.
- Extensive built-in libraries.
- Excellent visualization capabilities.
- Strong support for engineering and scientific applications.

Limitations of MATLAB

- Requires a paid license for most users.
- Can be slower than compiled languages like C or C++ for some tasks.
- Less suitable for building general-purpose desktop or web applications.

Basic MATLAB Commands

Command	Description
clc	Clears the Command Window
clear	Removes variables from the workspace
close all	Closes all figure windows
who	Lists current variables
whos	Displays detailed variable information
help command	Shows help for a command
Command	Purpose
zeros(m,n)	Creates an m × n zero matrix
ones(m,n)	Creates an m × n matrix of ones
eye(n)	Creates an n × n identity matrix
rand(m,n)	Creates an m × n random matrix
size(A)	Returns the dimensions of matrix A
length(A)	Returns the length of the largest dimension
det(A)	Computes the determinant
inv(A)	Computes the inverse of a matrix
eig(A)	Computes the eigenvalues
A'	Computes the transpose of the matrix
A*B	Performs matrix multiplication
A+B	Performs matrix addition
A-B	Performs matrix subtraction

Example Program with User Input:

```
clc;
clear;
close all;

% Read the order of the matrix
n = input('Enter the order of the square matrix: ');

% Read matrix elements
A = input('Enter the matrix in MATLAB format: ');
   disp(A);
```

Result/Output is: Matrix A

EXPERIMENT – 1

Finding Real and Complex Eigenvalues of a Matrix Using MATLAB Script with User Input

1.1 Aim:

To write and execute a MATLAB script to find the **real and complex eigenvalues of a square matrix**

1.2 Theory:

For a square matrix A, the eigenvalues are obtained from the characteristic equation
$\det(A-\lambda I)=0$
where:

- A = given square matrix
- λ = eigenvalue
- I = identity matrix

MATLAB provides the function **eig(A)** to calculate the eigenvalues of matrix A.

1.3 Algorithm:

1. Start MATLAB.
2. Clear the Command Window and Workspace.
3. Ask the user to enter the order of the matrix.
4. Ask the user to enter the matrix.
5. Use eig(A) to calculate the eigenvalues.
6. Display the eigenvalues.
7. Stop.

1.4 MATLAB Program:

```
clc;
clear;
close all;

% Read the order of the matrix
n = input('Enter the order of the matrix: ');

% Read the matrix from the user
A = input('Enter the matrix A: ');

% Check whether the matrix is square
[r,c] = size(A);
if r~= c
   disp('Error: Matrix must be square.');
else
   % Find eigenvalues
   eigenvalues = eig(A);
```

```
    % Display the matrix
    disp('Given Matrix A = ');
    disp(A);

    % Display eigenvalues
    disp('Eigenvalues of the matrix are:');
    disp(eigenvalues);
end
```

1.5 Sample Input:

Suppose the user enters:
Enter the order of the matrix: 3
Enter the matrix A: [4 1 0; 1 4 1; 0 1 4]

Output:

Given Matrix A =

4	1	0
1	4	1
0	1	4

Eigenvalues of the matrix are:

2.5858
4.0000
5.4142

1.6 Result:

Thus, the MATLAB script was successfully executed to find the **eigenvalues of a user-input square matrix**.

1.7 Viva Questions:

1. What is an eigenvalue?
2. What is an eigenvector?
3. What MATLAB function is used to find eigenvalues?
4. What is the characteristic equation?
5. Why must the matrix be square?
6. What is the use of input() in MATLAB?
7. What does eig(A) return?

EXPERIMENT – 2

Finding Eigenvalues and Eigenvectors of a Matrix Using MATLAB

2.1 Aim:

To write and execute a MATLAB program to find the **eigenvalues and corresponding eigenvectors of a given square matrix**.

2.2 Theory:

For a square matrix A, an eigenvalue λ and its corresponding eigenvector X satisfy
$AX=\lambda X$ or $(A-\lambda I)X=0$
For a non-zero vector X, $\det(A-\lambda I)=0$
The roots of this characteristic equation are the eigenvalues.

MATLAB provides [V,D] = eig(A)
where:

- D contains the eigenvalues on its main diagonal.
- V contains the corresponding eigenvectors as columns.

2.3 Algorithm:

1. Start MATLAB.
2. Define the given square matrix A.
3. Use [V,D] = eig(A) to calculate eigenvalues and eigenvectors.
4. Extract the eigenvalues using diag(D).
5. Display the eigenvalues.
6. Display the corresponding eigenvectors.
7. Verify the result using AV=VD.
8. Stop.

2.4 MATLAB Program:

```
clc;
clear;
close all;

% Read the order of the matrix
n = input('Enter the order of the matrix: ');

% Read the matrix from the user
A = input('Enter the matrix A: ');

% Check whether the matrix is square
[r,c] = size(A);
if r~= c
   disp('Error: Matrix must be square.');
```

```
else
% Find eigenvalues and eigenvectors
[V,D] = eig(A);

% Extract eigenvalues
eigenvalues = diag(D);

% Display the matrix
disp('Given Matrix A = ');
disp(A);

% Display eigenvalues
disp('Eigenvalues = ');
disp(eigenvalues);

% Display eigenvectors
disp('Corresponding Eigenvectors = ');
disp(V);

% Verification
disp('Verification of A*V = V*D');

disp('A*V = ');
disp(A*V);

disp('V*D = ');
disp(V*D);
end
```

2.5 Result

Thus, the **eigenvalues and corresponding eigenvectors of the given matrix were successfully calculated using MATLAB** and verified using

2.6 Input Matrix

A=[4 1 0;1 4 1;0 1 4]

Output

```
Given Matrix A =

     4     1     0
     1     4     1
     0     1     4

Eigenvalues =
    2.5858
    4.0000
    5.4142
```

Corresponding eigenvectors are displayed in matrix V:
Approximately,

$$V = \begin{bmatrix} 0.5000 & -0.7071 & 0.5000 \\ -0.7071 & 0 & 0.7071 \\ 0.5000 & 0.7071 & 0.5000 \end{bmatrix}$$

2.7 Viva Questions

1. Define eigenvalue.
2. Define eigenvector.
3. What is the MATLAB command for eigenvalues?
4. What is the MATLAB command for eigenvalues and eigenvectors?
5. What does `V` represent in `[V,D] = eig(A)`?
6. What does `D` represent?
7. How are eigenvalues extracted from `D`?
8. What is the characteristic equation?
9. How do you verify the eigenvectors?
10. What is the relation between A, V, and D?

2.8 Difference Between Experiment 1 and Experiment 2

Experiment	Input	MATLAB Function	Output
1	User enters matrix	`eig(A)`	Eigenvalues only
2	Matrix is given in program	`[V,D]=eig(A)`	Eigenvalues + Eigenvectors

EXPERIMENT – 3

Finding Root of a Given Equation Using Bisection Method

3.1. Aim

To write and execute a **MATLAB script** to find the approximate root of a given algebraic or transcendental equation using the **Bisection Method**.

3.2. Objective

After completing this experiment, students will be able to:

- Understand the concept of the Bisection Method.
- Find a root of an equation within a specified interval.
- Implement the Bisection Method using MATLAB.
- Generate an iteration table.
- Determine the approximate root for a specified tolerance.

3.3. Theory

The **Bisection Method** is a numerical method used to find the root of an equation: $f(x)=0$. Suppose we have two values a and b such that: $f(a) \times f(b)<0$. This means that the function changes its sign between a and b, so at least one root lies in the interval [a,b], provided f(x) is continuous. The midpoint of the interval is calculated using: $c=(a+b)/2$. The value of f(c) is then checked.

Case 1

If $f(a)f(c)<0$ the root lies between a and c. Therefore,b=c

Case 2

If $f(a)f(c)>0$ the root lies between c and b.Therefore,a=c This process is repeated until the desired accuracy is obtained.

3.4. Algorithm

Step 1: Define the function f(x).
Step 2: Select two initial values a and b.
Step 3: Verify: $f(a)f(b)<0$. If this condition is not satisfied, choose another interval.
Step 4: Calculate the midpoint: : $c=(a+b)/2$
Step 5: Calculate f(c).
Step 6: If $f(c)=0$, then c is the root.
Step 7: Otherwise, check the sign:

- If $f(a)f(c)<0$, set b=c.
- Otherwise, set a=c.

Step 8: Repeat Steps 4–7 until the required tolerance is obtained.
Step 9: Display the approximate root.

3.5. MATLAB Program

```
clc;
clear;
clear all;
% User enters the function
f = input('Enter the function f(x): ');
% User enters the initial values
a = input('Enter the lower limit a: ');
b = input('Enter the upper limit b: ');
% Check the condition
if f(a) * f(b) >= 0
    disp('Bisection method cannot be applied.');
    disp('Choose a and b such that f(a)*f(b) < 0.');
else
    disp('Iteration        a        b        c       f(c)');

    for i = 1:100
        c = (a + b)/2;

        fprintf('%d\t\t %.4f\t %.4f\t %.4f\t %.4f\n', i, a, b, c, f(c));

        if f(a)*f(c) < 0
            b = c;
        else
            a = c;
        end
    end

    fprintf('\nRoot = %.4f\n', c);
end
```

Sample inputs:

```
% 1 Enter the function f(x): @(x) x^3-x-2
% Enter the lower limit a: 1
% Enter the upper limit b: 2

% 2 Exponential function
% Enter the function f(x): @(x) exp(x) - 3*x
% Enter the lower limit a: 0
% Enter the upper limit b: 1

% 3 Logarithmic function
% Enter the function f(x):@(x) log(x) - x + 2
% Enter the lower limit a: 0
% Enter the upper limit b: 1
```

3.6 Results

3.7 Viva Questions:

1. What is the Bisection Method?
2. Condition for Bisection?

3. What is the Bisection formula?
4. Does Bisection require derivative?

Input:

Enter the function f(x): @(x) x^3-x-2
Enter the lower limit a: 1
Enter the upper limit b: 2
Enter the tolerance: 0.000001

Examples:

S.No	Equation	MATLAB input
1	(x^3-x-2)	@(x) x^3-x-2
2	(x^2-4)	@(x) x^2-4
3	(x^3-4x-9)	@(x) x^3-4*x-9
4	(\sin x-x/2)	@(x) sin(x)-x/2
5	(e^x-3x)	@(x) exp(x)-3*x

MATLAB Command	Purpose
`clc`	Clears Command Window
`clear`	Clears variables
`close all`	Closes all figures
`@(x)`	Creates an anonymous function
`disp or fprintf()`	Displays formatted output
`while`	Repeats statements while condition is true
`for`	Executes statements repeatedly
`abs()`	Finds absolute value

EXPERIMENT – 4

Root of a Given Equation Using Newton–Raphson Method

4.1. Aim

To write and execute a **MATLAB script** to find the approximate root of a given algebraic or transcendental equation using the **Newton–Raphson Method**.

4.2. Objective

After completing this experiment, students will be able to:

- Understand the Newton–Raphson Method.
- Calculate the derivative of a given function.
- Find the root using an initial approximation.
- Implement the method using MATLAB.
- Understand the convergence of an iterative numerical method.

4.3. Theory

The **Newton–Raphson Method** is an iterative numerical method for finding the root of: $f(x)=0$
The method uses the tangent to the curve at the current approximation. The Newton–Raphson formula is:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

where:

- x_n = current approximation
- x_{n+1} = next approximation
- $f(x_n)$ = function value
- $f'(x_n)$ = derivative of the function

The process continues until: $|x_{n+1}-x_n|<\epsilon$ where ϵ is the specified tolerance.

4.4. Algorithm

Step 1: Define the function f(x).
Step 2: Define its derivative f′(x).
Step 3: Select an initial approximation x0.
Step 4: Calculate the next approximation using:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Step 5: Check the convergence condition: $|x_{n+1}-x_n|<\epsilon$
Step 6: If the condition is satisfied, stop the iteration.
Step 7: Otherwise, set: $x_n=x_{n+1}$ and repeat.
Step 8: Display the approximate root.

4.5 MATLAB Script

```
clc;
clear;

f = input('Enter f(x): ');
df = input('Enter f''(x): ');
x = input('Enter initial value: ');
```

```
tol = input('Enter tolerance: ');

fprintf('\nIter\t x\t\t f(x)\n');

for i = 1:100
    xi = x - f(x)/df(x);
    fprintf('%d\t %.6f\t %.6f\n',i,xi,f(xi));

    if abs(xi-x) < tol
        break;
    end
    x = xi;
end

fprintf('\nRoot = %.6f\n',xi);
fprintf('Iterations = %d\n',i);
```

Sample Inputs

Example-1
Enter the function f(x): @(x) x^2-4
Enter the derivative f'(x): @(x) 2*x
Enter the initial approximation x0: 3
Enter the tolerance: 0.0001

No.	Enter f(x)	Enter f'(x)	Initial value	Tolerance
1	@(x) x^3-x-2	@(x) 3*x^2-1	1.5	0.0001
2	@(x) x^2-4	@(x) 2*x	3	0.0001
3	@(x) x^3-5*x+1	@(x) 3*x^2-5	2	0.0001
4	@(x) x^2-5	@(x) 2*x	2	0.0001
5	@(x) x^3-2*x-5	@(x) 3*x^2-2	2	0.0001
6	@(x) x^3+x-1	@(x) 3*x^2+1	0.5	0.0001
7	@(x) x^2-3*x-4	@(x) 2*x-3	4	0.0001
8	@(x) x^3-4*x-9	@(x) 3*x^2-4	2.5	0.0001
9	@(x) x^4-x-10	@(x) 4*x^3-1	2	0.0001
10	@(x) x^3-6*x^2+11*x-6	@(x) 3*x^2-12*x+11	3.5	0.0001

4.6 Result:

4.7 Viva Questions

1. What is Newton–Raphson formula?
2. Does Newton–Raphson require derivative?
3. What is x_0?
4. Which method is generally faster?
5. What is tolerance?

Difference between Bisection and Newton–Raphson

Bisection	Newton–Raphson
Uses two initial values (a,b)	Uses one initial value (x_0)
Does not require derivative	Requires derivative
Generally slower	Generally faster
More reliable	Depends on initial guess
Uses interval halving	Uses tangent approximation

EXPERIMENT – 5
Solution of Linear Equations Using Jacobi's Iterative Method

5.1. Aim
To write and execute a MATLAB script to find the solution of a system of simultaneous linear equations using **Jacobi's Iterative Method**.

5.2. Objective
After completing this experiment, students will be able to:
- Understand the Jacobi Iterative Method.
- Solve a system of simultaneous linear equations iteratively.
- Understand the concept of initial approximation.
- Implement Jacobi's Method using MATLAB.
- Understand convergence in an iterative numerical method.

5.3. Theory

The **Jacobi Iterative Method** is an iterative numerical method used to solve a system of linear equations.
Consider the system:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3 \end{aligned} \quad \Rightarrow \quad \begin{aligned} x_1 &= (b_1 - a_{12}x_2 - a_{13}x_3)/a_{11} \\ x_2 &= (b_2 - a_{21}x_1 - a_{23}x_3)/a_{22} \\ x_3 &= (b_3 - a_{31}x_1 - a_{32}x_2)/a_{33} \end{aligned}$$

Step 1: Construct updating formula as

$$\begin{aligned} x_1^{(i+1)} &= (b_1 - a_{12}x_2^{(i)} - a_{13}x_3^{(i)})/a_{11} \\ x_2^{(i+1)} &= (b_2 - a_{21}x_1^{(i)} - a_{23}x_3^{(i)})/a_{22} \\ x_3^{(i+1)} &= (b_3 - a_{31}x_1^{(i)} - a_{32}x_2^{(i)})/a_{33} \end{aligned}$$

Step 2: Starts with initial guesses $x_1^{(0)}, x_2^{(0)}, x_3^{(0)}$

Step 3: Iteratively compute $x_1^{(i+1)}, x_2^{(i+1)}, x_3^{(i+1)}$ based on the updating formula for $i = 0, 1, \ldots$, until the x's converge

For the system:

$$10x_1 - x_2 + 2x_3 = 6$$
$$-x_1 + 11x_2 - x_3 = 25$$
$$2x_1 - x_2 + 10x_3 = -11$$

we first rearrange:

$$x_1 = \frac{6 + x_2 - 2x_3}{10}$$
$$x_2 = \frac{25 + x_1 + x_3}{11}$$
$$x_3 = \frac{-11 - 2x_1 + x_2}{10}$$

Starting with:

$$x_1^{(0)} = 0, \quad x_2^{(0)} = 0, \quad x_3^{(0)} = 0$$

Iteration 1

$$x_1^{(1)} = \frac{6 + 0 - 2(0)}{10} = 0.6000$$
$$x_2^{(1)} = \frac{25 + 0 + 0}{11} = 2.2727$$
$$x_3^{(1)} = \frac{-11 - 2(0) + 0}{10} = -1.1000$$

Therefore,

$$\boxed{(x_1, x_2, x_3) = (0.6000,\ 2.2727,\ -1.1000)}$$

Iteration 2

Use only Iteration 1 values:

$$x_1^{(2)} = \frac{6 + 2.2727 - 2(-1.1000)}{10}$$
$$= \frac{10.4727}{10} = \boxed{1.04727}$$
$$x_2^{(2)} = \frac{25 + 0.6000 - 1.1000}{11}$$
$$= \frac{24.5}{11} = \boxed{2.22727}$$
$$x_3^{(2)} = \frac{-11 - 2(0.6000) + 2.2727}{10}$$
$$= \frac{-9.9273}{10} = \boxed{-0.99273}$$

So,

$$\boxed{(x_1, x_2, x_3) = (1.04727,\ 2.22727,\ -0.99273)}$$

Iteration 3

$$x_1^{(3)} = \frac{6 + 2.22727 - 2(-0.99273)}{10} = 1.021273$$
$$x_2^{(3)} = \frac{25 + 1.04727 - 0.99273}{11} = 2.277686$$
$$x_3^{(3)} = \frac{-11 - 2(1.04727) + 2.22727}{10} = -1.086727$$
$$\boxed{(x_1, x_2, x_3) = (1.021273,\ 2.277686,\ -1.086727)}$$

5.4. Algorithm

Step 1: Define the coefficient matrix A.
Step 2: Define the constant matrix b.
Step 3: Select initial approximations for x1,x2,…,xn
Step 4: Rearrange the equations to calculate each variable.
Step 5: Calculate the new values using the previous iteration values.
Step 6: Repeat the calculation for the specified number of iterations.
Step 7: Check whether the values are converging.
Step 8: Display the approximate solution.

5.5. MATLAB Script

```
clc;
clear;

A = [10 -1 2; -1 11 -1; 2 -1 10];
b = [6; 25; -11];

x = [0; 0; 0];

fprintf('Iter\t x\t\t y\t\t z\n');

for k = 1:20
   x1 = (6 + x(2) - 2*x(3))/10;
   x2 = (25 + x(1) + x(3))/11;
   x3 = (-11 - 2*x(1) + x(2))/10;

   x = [x1; x2; x3];

   fprintf('%d\t %.6f\t %.6f\t %.6f\n', k, x1, x2, x3);
end

fprintf('\nSolution:\n');
disp(x);
```

5.6 Results

5.7. Viva Questions

1. **What is Jacobi's Method?**
   Jacobi's Method is an iterative technique for solving simultaneous linear equations.
2. **What is the initial approximation?**
   It is the assumed starting value of the unknown variables.
3. **Does Jacobi's Method use old or new values?**
   It uses the **old values** from the previous iteration to calculate all new values.
4. **When does Jacobi's Method converge?**
   It generally converges when the coefficient matrix satisfies suitable conditions such as **strict diagonal dominance**.
5. **What is an iteration?**
   One complete calculation of the new values of all unknown variables is called an iteration.

Sample Inputs:

No.	Coefficient Matrix AA	Constant Matrix bb	Initial Value	Iterations
1	[10 -1 2; -1 11 -1; 2 -1 10]	[6; 25; -11]	[0; 0; 0]	20
2	[10 -1 2; -1 11 -1; 2 -1 10]	[6; 25; -11]	[1; 1; 1]	20
3	[8 1 1; 1 9 -1; 1 -1 7]	[7; 8; 6]	[0; 0; 0]	20
4	[10 1 1; 2 10 1; 1 2 10]	[12; 13; 13]	[0; 0; 0]	20
5	[12 -1 2; 1 10 -1; 2 -1 12]	[15; 10; 20]	[0; 0; 0]	20
6	[15 1 1; 1 12 -1; 2 1 10]	[17; 12; 13]	[0; 0; 0]	20
7	[10 -2 1; 1 10 -1; 1 -1 8]	[9; 10; 8]	[0; 0; 0]	20
8	[12 1 2; 2 15 -1; 1 1 10]	[15; 20; 12]	[0; 0; 0]	20
9	[10 -1 1; 1 11 2; 2 1 12]	[10; 15; 20]	[0; 0; 0]	20
10	[11 1 1; 1 10 -1; 1 -1 9]	[13; 10; 9]	[0; 0; 0]	20

EXPERIMENT – 6

Solution of a System of Linear Equations Using Gauss–Seidel Method

6.1. Aim

To write and execute a MATLAB script to find the solution of a system of simultaneous linear equations using the **Gauss–Seidel Iterative Method**.

6.2. Objective

After completing this experiment, students will be able to:

- Understand the Gauss–Seidel Iterative Method.
- Solve a system of simultaneous linear equations iteratively.
- Understand the use of initial approximations.
- Implement Gauss–Seidel Method using MATLAB.
- Understand convergence in an iterative numerical method.

6.3. Theory

The **Gauss–Seidel Method** is an iterative numerical technique used to solve a system of simultaneous linear equations.

For a system of equations, each variable is rearranged as:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)}}{a_{ii}}$$

The important feature of Gauss–Seidel Method is that **the newly calculated value is immediately used** in calculating the next variable.

For example:

$$10x - y + 2z = 6$$
$$-x + 11y - z = 25$$
$$2x - y + 10z = -11$$

Rearranging:

$$x = \frac{6 + y - 2z}{10}$$
$$y = \frac{25 + x + z}{11}$$
$$z = \frac{-11 - 2x + y}{10}$$

Unlike Jacobi's method, Gauss–Seidel uses the **new value of** x while calculating y, and the **new values of** x **and** y while calculating z.

6.4. Algorithm

Step 1: Define the coefficient matrix AA.

Step 2: Define the constant matrix bb.

Step 3: Select initial values for the unknown variables.

Step 4: Rearrange each equation to calculate the corresponding variable.

Step 5: Calculate the new value of x1x_1.

Step 6: Use the new x1x_1 to calculate x2x_2.

Step 7: Use the new x1x_1 and x2x_2 to calculate x3x_3.

Step 8: Repeat the process until the required number of iterations is completed or convergence is achieved.

Step 9: Display the approximate solution.

6.5. MATLAB Script

```
clc;
clear;

A = [10 -1 2; -1 11 -1; 2 -1 10];
b = [6; 25; -11];
x = [0; 0; 0];

for k = 1:10
    x(1) = (b(1)-A(1,2)*x(2)-A(1,3)*x(3))/A(1,1);
    x(2) = (b(2)-A(2,1)*x(1)-A(2,3)*x(3))/A(2,2);
    x(3) = (b(3)-A(3,1)*x(1)-A(3,2)*x(2))/A(3,3);

    fprintf('Iter %d: %.6f %.6f %.6f\n',k,x);
end
fprintf('\nSolution:\n');
disp(x);
```

Manual Calculation – Example 1

Given:

$$10x - y + 2z = 6$$

$$-x + 11y - z = 25$$

$$2x - y + 10z = -11$$

Rearranging:

$$x = \frac{6 + y - 2z}{10}$$

$$y = \frac{25 + x + z}{11}$$

$$z = \frac{-11 - 2x + y}{10}$$

Initial values:

$$x^{(0)} = 0, \quad y^{(0)} = 0, \quad z^{(0)} = 0$$

Iteration 1

Calculate x:

$$x^{(1)} = \frac{6 + 0 - 2(0)}{10} = 0.6000$$

Now use the **new** $x^{(1)}$ to calculate y:

$$y^{(1)} = \frac{25 + 0.6000 + 0}{11} = 2.32727$$

Now use the **new** $x^{(1)}$ **and** $y^{(1)}$ to calculate z:

$$z^{(1)} = \frac{-11 - 2(0.6000) + 2.32727}{10} = -0.98727$$

Therefore:

$$\boxed{x^{(1)} = 0.6000, \quad y^{(1)} = 2.32727, \quad z^{(1)} = -0.98727}$$

Iteration 2

$$x^{(2)} = \frac{6 + 2.32727 - 2(-0.98727)}{10} = 1.03018$$

$$y^{(2)} = \frac{25 + 1.03018 - 0.98727}{11} = 2.27663$$

$$z^{(2)} = \frac{-11 - 2(1.03018) + 2.27663}{10} = -1.07837$$

Therefore:

$$\boxed{x^{(2)} = 1.03018, \quad y^{(2)} = 2.27663, \quad z^{(2)} = -1.07837}$$

6.6. Result

The given system of linear equations was successfully solved using the **Gauss–Seidel Iterative Method** in MATLAB.

The approximate solution is:

x=1.04327,y=2.26923,z=−1.08173

6.7. Viva Questions

1. **What is Gauss–Seidel Method?**
   It is an iterative method used to solve simultaneous linear equations.
2. **What is the main difference between Jacobi and Gauss–Seidel methods?**
   Jacobi uses old values for all variables, whereas Gauss–Seidel immediately uses newly calculated values.
3. **What is an initial approximation?**
   It is the assumed starting value of the unknown variables.
4. **When does Gauss–Seidel Method converge?**
   It generally converges when the coefficient matrix satisfies suitable conditions such as strict diagonal dominance.
5. **Why is Gauss–Seidel usually faster than Jacobi?**
   Because it uses the latest calculated values during the same iteration.

No.	Coefficient Matrix AA	Constant Matrix bb	Initial Value	Iterations
1	[10 -1 2; -1 11 -1; 2 -1 10]	[6; 25; -11]	[0; 0; 0]	20
2	[8 1 1; 1 9 -1; 1 -1 7]	[7; 8; 6]	[0; 0; 0]	20
3	[10 1 1; 2 10 1; 1 2 10]	[12; 13; 13]	[0; 0; 0]	20
4	[12 -1 2; 1 10 -1; 2 -1 12]	[15; 10; 20]	[0; 0; 0]	20
5	[15 1 1; 1 12 -1; 2 1 10]	[17; 12; 13]	[0; 0; 0]	20
6	[10 -2 1; 1 10 -1; 1 -1 8]	[9; 10; 8]	[0; 0; 0]	20
7	[12 1 2; 2 15 -1; 1 1 10]	[15; 20; 12]	[0; 0; 0]	20
8	[10 -1 1; 1 11 2; 2 1 12]	[10; 15; 20]	[0; 0; 0]	20
9	[11 1 1; 1 10 -1; 1 -1 9]	[13; 10; 9]	[0; 0; 0]	20
10	[14 -1 2; 1 12 -1; 2 -1 15]	[15; 12; 20]	[0; 0; 0]	20

EXPERIMENT – 7

Solving Exact and Non-Exact Differential Equations

Aim

To determine whether a given first-order differential equation is **exact or non-exact** and obtain its solution when it is exact using MATLAB.

Theory

A first-order differential equation is written in the form

$$M(x,y)\,dx + N(x,y)\,dy = 0$$

The equation is called **exact** if

$$\boxed{\frac{\partial M}{\partial y} = \frac{\partial N}{\partial x}}$$

If

$$\boxed{\frac{\partial M}{\partial y} \neq \frac{\partial N}{\partial x}}$$

then the equation is called **non-exact**.

For an exact equation,

$$M\,dx + N\,dy = 0$$

there exists a function $F(x,y)$ such that

$$\frac{\partial F}{\partial x} = M$$

and

$$\frac{\partial F}{\partial y} = N$$

Therefore, the solution is

$$\boxed{F(x,y) = C}$$

Given Equation

Consider

$$(2xy + 3)\,dx + (x^2 + 4y)\,dy = 0$$

Here,

$$M = 2xy + 3$$

and

$$N = x^2 + 4y$$

Now,

$$\frac{\partial M}{\partial y} = 2x$$

and

$$\frac{\partial N}{\partial x} = 2x$$

Therefore,

$$\frac{\partial M}{\partial y} = \frac{\partial N}{\partial x}$$

Hence, the equation is exact.

Algorithm

1. Start MATLAB.
2. Define the symbolic variables x and y.
3. Define M(x,y) and N(x,y)..
4. Find My using `diff(M,y)`.
5. Find Nx using `diff(N,x)`.
6. Compare My and Nx.
7. If they are equal, the equation is exact.
8. Integrate M with respect to x.
9. Determine the remaining function of y.
10. Obtain the general solution.
11. Display the result.

MATLAB Program

```
clc;
clear;

syms x y C

% Define M(x,y) and N(x,y)
M = 2*x*y + 3;
N = x^2 + 4*y;

% Find partial derivatives
My = diff(M,y);
Nx = diff(N,x);
```

```
disp('M(x,y) = ');
disp(M);

disp('N(x,y) = ');
disp(N);

disp('Partial derivative of M with respect to y = ');
disp(My);

disp('Partial derivative of N with respect to x = ');
disp(Nx);

% Check whether equation is exact
if simplify(My-Nx) == 0

    disp('The differential equation is EXACT.');

    % Integrate M with respect to x
    F = int(M,x);

    % Find remaining function of y
    Gprime = simplify(N-diff(F,y));

    G = int(Gprime,y);

    % Complete solution
    F = simplify(F+G);

    disp('General solution is:');
    disp(F == C);

else

    disp('The differential equation is NON-EXACT.');

end
```

Output

M(x,y) =
2*x*y + 3
N(x,y) =
x^2 + 4*y
Partial derivative of M with respect to y =
2*x
Partial derivative of N with respect to x =
2*x
The differential equation is EXACT.
General solution is:
x^2*y + 3*x + 2*y^2 = C

Q.No.	Practice Problem
1	Determine whether $(2xy+3)dx+(x^2+4y)dy=0$ is exact. If exact, find the solution.
2	Determine whether $(3x^2+2y)dx+(2x+4y^3)dy=0$ is exact. Find the general solution.
3	Test whether $(y+2x)dx+(x+3y^2)dy=0$ is exact.
4	Check whether $(2xy+4x)dx+(x^2+2y)dy=0$ is exact and solve it.
5	Determine whether $(2xy+3y)dx+(x^2+4x)dy=0$ is exact or non-exact.
6	Check the exactness of $(3x^2y+2y)dx+(x^3+2x+4y)dy=0$.
7	Determine whether $(2x+y)dx+(x+2y)dy=0$ is exact.
8	Test whether $(y\cos x+2x)dx+(\sin x+2y)dy=0$ is exact.
9	Check whether $(2xy+y^2)dx+(x^2+2xy)dy=0$ is exact and find its solution.
10	Write a MATLAB program to check whether a given differential equation $Mdx+Ndy=0$ is exact or non-exact.

EXPERIMENT – 8

Exponential Growth and Decay

Aim

To solve an **exponential growth/decay problem** using MATLAB.

Theory

If the rate of change of a quantity is proportional to the quantity itself, then

dy/dt=ky

where k is the proportionality constant.

Exponential Growth

For $k > 0$,

$$\boxed{y = y_0 e^{kt}}$$

represents exponential growth.

Exponential Decay

For $k < 0$,

$$\boxed{y = y_0 e^{-kt}}$$

Problem

The initial population of a town is **500**. The population grows at a rate proportional to the population with growth constant k=0.02 Find the population after **10 years**.

Algorithm

1. Start MATLAB.
2. Enter the initial value y_0.
3. Enter the growth/decay constant k.
4. Enter the required time t.
5. Use the formula $y=y_0e^{kt}$.
6. Calculate the value using MATLAB `exp()`.
7. Display the result.

MATLAB Program

```
clc;
clear;

% Initial population
y0 = 500;

% Growth constant
k = 0.02;

% Time
t = 10;

% Exponential growth
y = y0 * exp(k*t);

fprintf('Initial population = %.2f\n', y0);
fprintf('Growth constant = %.2f\n', k);
fprintf('Time = %.2f years\n', t);
fprintf('Population after %.2f years = %.4f\n', t, y);
```

Output

Initial population = 500.00
Growth constant = 0.02
Time = 10.00 years
Population after 10.00 years = 610.7014

Exponential Decay Example

Problem

A radioactive substance initially has a mass of **100 mg**. It decays according to dy/dt=−0.05y Find the amount remaining after 10 units of time.

```
clc;
clear;

y0 = 100;
k = 0.05;
t = 10;

y = y0 * exp(-k*t);

fprintf('Initial amount = %.2f mg\n', y0);
fprintf('Decay constant = %.2f\n', k);
fprintf('Time = %.2f\n', t);
fprintf('Amount remaining = %.4f mg\n', y);
```

Output

```
Initial amount = 100.00 mg
Decay constant = 0.05
Time = 10.00
Amount remaining = 60.6531 mg
```

Q.No.	Practice Problem
1	A population is initially 1000 and grows according to $\frac{dP}{dt} = 0.03P$. Find the population after 5 years.
2	A population of 20,000 grows at a rate of 4% per year. Find its population after 8 years.
3	A quantity initially equal to 500 mg decays according to $\frac{dN}{dt} = -0.08N$. Find the quantity after 6 hours.
4	A radioactive substance has an initial mass of 200 g and decay constant 0.1. Find its mass after 10 years.
5	A population increases from 500 to 800 in 10 years. Find the growth constant k.
6	A bacterial culture contains 200 bacteria initially and doubles every 3 hours. Find the number of bacteria after 12 hours.
7	A substance has an initial amount of 1000 g and decreases at a rate of 5% per hour. Find the amount after 10 hours.
8	A population is 5000 initially and becomes 8000 after 6 years. Find the growth constant and population after 10 years.
9	A radioactive material has a half-life of 5 years. If the initial quantity is 800 mg, find the quantity after 15 years.

EXPERIMENT – 9

Newton's Law of Cooling

Aim

To determine the temperature of an object at a specified time using **Newton's Law of Cooling** and MATLAB.

Theory

Newton's Law of Cooling states that the rate of change of temperature of an object is proportional to the difference between the temperature of the object and the surrounding temperature.

Mathematically,

$dT/dt = -k(T - Ts)$

where

- T = temperature of the object at time tt
- Ts = surrounding temperature
- T_0 = initial temperature
- k = cooling constant
- t = time

The solution is

$T = Ts + (T_0 - Ts)e^{-kt}$

Problem

A cup of coffee initially has a temperature of 90^0CThe surrounding room temperature is 25^0 C After 10 minutes, the temperature of the coffee is70^0 CFind the temperature of the coffee after 20 minutes.

Algorithm

1. Start MATLAB.
2. Enter the initial temperature T_0.
3. Enter the surrounding temperature Ts.
4. Enter the known temperature T_1.
5. Enter the corresponding time t_1.

6. Calculate the cooling constant k.
7. Enter the required time.
8. Apply Newton's cooling formula.
9. Calculate the temperature.
10. Display the result.

MATLAB Program

```
clc;
clear;

% Given data
T0 = 90;       % Initial temperature
Ts = 25;       % Surrounding temperature
T1 = 70;       % Temperature after 10 minutes
t1 = 10;       % Time corresponding to T1

% Calculate cooling constant
k = -(1/t1) * log((T1-Ts)/(T0-Ts));

fprintf('Cooling constant k = %.5f\n', k);

% Required time
t = 20;

% Newton's law of cooling
T = Ts + (T0-Ts)*exp(-k*t);

fprintf('Temperature after %d minutes = %.4f degree C\n', ...
    t, T);
```

Q.No.	Practice Problem
1	A body initially at $100°C$ is placed in a room at $20°C$. After 10 minutes its temperature is $60°C$. Find its temperature after 20 minutes.
2	A body initially at $90°C$ is kept in surroundings at $25°C$. After 5 minutes its temperature is $70°C$. Find its temperature after 15 minutes.
3	A body initially at $80°C$ is placed in a room at $30°C$. Its temperature becomes $50°C$ after 10 minutes. Find its temperature after 30 minutes.
4	A cup of coffee is initially at $90°C$ in a room at $20°C$. After 10 minutes its temperature is $60°C$. Find its temperature after 20 minutes.
Q.No.	Practice Problem
5	A heated object cools from $120°C$ to $80°C$ in 10 minutes in a room maintained at $20°C$. Find its temperature after another 10 minutes.
6	A body at $100°C$ is placed in surroundings at $25°C$. Its temperature is $75°C$ after 5 minutes. Find the temperature after 15 minutes.
7	A metal object initially at $150°C$ is placed in a room at $30°C$. After 10 minutes its temperature is $90°C$. Find the cooling constant.
8	A body cools from $80°C$ to $50°C$ in 10 minutes when the surrounding temperature is $20°C$. Find its temperature after 20 minutes.
9	A body initially at $100°C$ cools to $70°C$ in 10 minutes in a room at $25°C$. Find its temperature after 30 minutes.
10	A hot object initially at $200°C$ is placed in surroundings at $25°C$. After 20 minutes its temperature is $100°C$. Find its temperature after 40 minutes.

Important formulas for practice

Experiment	Formula
Exact Equation	$M(x,y)dx + N(x,y)dy = 0$
Exactness Condition	$\frac{\partial M}{\partial y} = \frac{\partial N}{\partial x}$
Exponential Growth	$P = P_0 e^{kt}$
Exponential Decay	$N = N_0 e^{-kt}$

Experiment	Formula
Growth Constant	$k = \frac{1}{t} \ln\left(\frac{P}{P_0}\right)$
Decay Constant	$k = \frac{1}{t} \ln\left(\frac{N_0}{N}\right)$
Newton's Cooling Law	$\frac{dT}{dt} = -k(T - T_s)$
Cooling Formula	$T = T_s + (T_0 - T_s)e^{-kt}$

MATLAB General Viva Questions

No.	Question
1	What is MATLAB?
2	What is the purpose of `clc`?
3	What is the purpose of `clear`?
4	What is the use of `disp()`?
5	What is the use of `fprintf()`?
6	What is the difference between `disp()` and `fprintf()`?
7	What is the use of `syms`?
8	What is the use of `diff()`?
9	What is the use of `int()`?
10	What is the use of `simplify()`?
11	What is the use of `exp()`?
12	What is the use of `log()`?
13	What is a symbolic variable?
14	What is the difference between symbolic and numerical calculations?
15	Why are MATLAB programs useful for solving differential equations?

EXPERIMENT – 10

Solution of Homogeneous Differential Equations Using MATLAB

1. Aim

To write a MATLAB program to solve higher-order linear homogeneous differential equations with constant coefficients.

2. Software Required

MATLAB with Symbolic Math Toolbox.

3. Theory

A linear homogeneous differential equation with constant coefficients is represented as

$$a_n y^{(n)} + a_{n-1} y^{(n-1)} + \cdots + a_1 y' + a_0 y = 0$$

The auxiliary equation is obtained by replacing the derivatives with powers of \(m\). Solving this equation gives the roots used to construct the general solution.

4. Algorithm

1. Start the MATLAB program.
2. Clear the command window and workspace.
3. Define the dependent variable symbolically.
4. Enter the homogeneous differential equation.
5. Use dsolve() to obtain the general solution.
6. Display the solution.
7. Stop.

5. MATLAB Program

```
clc;
clear;
close all;

% Homogeneous differential equation
% d2y/dx2 - 5*dy/dx + 6*y = 0

syms y(x)

% Define the differential equation
ode = diff(y,x,2) - 5*diff(y,x) + 6*y == 0;

% Solve the equation
sol = dsolve(ode);

disp('General solution is:');
disp(sol);
```

6. Sample Output

Solution of Homogeneous ODE:

C1*exp(2*x) + C2*exp(3*x)

7. Result

Thus, the given homogeneous differential equation was solved using MATLAB.

$$y = C_1 e^{2x} + C_2 e^{3x}$$

8. Viva Questions
    1. What is a homogeneous differential equation?
    2. What is an auxiliary equation?
    3. What is a complementary function?
    4. What is the purpose of dsolve()?
    5. What is the solution when the auxiliary equation has repeated roots?
    6. Which MATLAB toolbox is required for symbolic differential equation solving?

Solving homogeneous ODEs

No.	Differential equation	Auxiliary equation and roots	General solution
1	$y'' - 5y' + 6y = 0$	$m^2 - 5m + 6 = 0;\ m = 2, 3$	$y = C_1 e^{2x} + C_2 e^{3x}$
2	$y'' - 3y' + 2y = 0$	$m^2 - 3m + 2 = 0;\ m = 1, 2$	$y = C_1 e^{x} + C_2 e^{2x}$
3	$y'' - 4y' + 4y = 0$	$(m-2)^2 = 0$	$y = (C_1 + C_2 x) e^{2x}$
4	$y'' + 5y' + 6y = 0$	$(m+2)(m+3) = 0$	$y = C_1 e^{-2x} + C_2 e^{-3x}$
5	$y'' + 9y = 0$	$m^2 + 9 = 0;\ m = \pm 3i$	$y = C_1 \cos 3x + C_2 \sin 3x$
6	$y'' - 4y = 0$	$m^2 - 4 = 0;\ m = \pm 2$	$y = C_1 e^{2x} + C_2 e^{-2x}$
7	$y'' + 4y' + 4y = 0$	$(m+2)^2 = 0$	$y = (C_1 + C_2 x) e^{-2x}$
8	$y'' - y = 0$	$m^2 - 1 = 0;\ m = \pm 1$	$y = C_1 e^{x} + C_2 e^{-x}$
9	$y'' + 2y' + 5y = 0$	$m = -1 \pm 2i$	$y = e^{-x}(C_1 \cos 2x + C_2 \sin 2x)$
10	$y'' - 6y' + 9y = 0$	$(m-3)^2 = 0$	$y = (C_1 + C_2 x) e^{3x}$
11	$y'' + 7y' + 12y = 0$	$(m+3)(m+4) = 0$	$y = C_1 e^{-3x} + C_2 e^{-4x}$
12	$y'' + 16y = 0$	$m = \pm 4i$	$y = C_1 \cos 4x + C_2 \sin 4x$
13	$y''' - 6y'' + 11y' - 6y = 0$	$(m-1)(m-2)(m-3) = 0$	$y = C_1 e^{x} + C_2 e^{2x} + C_3 e^{3x}$
14	$y''' + 3y'' + 3y' + y = 0$	$(m+1)^3 = 0$	$y = (C_1 + C_2 x + C_3 x^2) e^{-x}$
15	$y''' - y' = 0$	$m(m-1)(m+1) = 0$	$y = C_1 + C_2 e^{x} + C_3 e^{-x}$
16	$y''' + 4y' = 0$	$m(m^2 + 4) = 0$	$y = C_1 + C_2 \cos 2x + C_3 \sin 2x$
17	$y''' - 3y'' + 3y' - y = 0$	$(m-1)^3 = 0$	$y = (C_1 + C_2 x + C_3 x^2) e^{x}$
18	$y''' - 4y'' + 5y' - 2y = 0$	$(m-1)^2(m-2) = 0$	$y = (C_1 + C_2 x) e^{x} + C_3 e^{2x}$
19	$y''' + y'' - 2y' = 0$	$m(m+2)(m-1) = 0$	$y = C_1 + C_2 e^{-2x} + C_3 e^{x}$
20	$y''' - 2y'' + 2y' - y = 0$	$(m-1)(m^2 - m + \downarrow = 0$	$y = C_1 e^{x} + e^{x/2}\left(C_2 \cos \frac{\sqrt{3}x}{2} + C_3 \sin \frac{\sqrt{3}x}{2}\right)$

EXPERIMENT – 11

Solution of Non-Homogeneous Differential Equations Using MATLAB

1. Aim

To write a MATLAB program to solve higher-order linear non-homogeneous differential equations with constant coefficients.

2. Software Required

MATLAB with Symbolic Math Toolbox.

3. Theory

A non-homogeneous differential equation is represented as

$$a_n y^{(n)} + \cdots + a_1 y' + a_0 y = X(x)$$

where $X(x) \neq 0$.

Its general solution is

$$\boxed{y = \text{C.F.} + \text{P.I.}}$$

Here, C.F. is the complementary function, and P.I. is the particular integral.

4. Algorithm

1. Start the MATLAB program.
2. Clear the command window and workspace.
3. Define the symbolic dependent variable.
4. Enter the non-homogeneous differential equation.
5. Use dsolve() to solve the equation.
6. Display the general solution.
7. Stop.

5. MATLAB Program

```
clc;
clear;
close all;

% Non-homogeneous differential equation
% d2y/dx2 - 3*dy/dx + 2*y = exp(3*x)

syms y(x)

% Define the differential equation
ode = diff(y,x,2) - 3*diff(y,x) + 2*y == exp(3*x);

% Solve the equation
sol = dsolve(ode);

disp('General solution is:');
disp(sol);
```

6. Sample Output

Solution of Non-Homogeneous ODE:

C1*exp(x) + C2*exp(2*x) + exp(3*x)/2

7. Result

Thus, the given non-homogeneous differential equation was solved using MATLAB.

$$\boxed{y = C_1 e^{x} + C_2 e^{2x} + \frac{1}{2} e^{3x}}$$

8. Viva Questions

1. What is a non-homogeneous differential equation?
2. What is a particular integral?
3. What is the difference between C.F. and P.I.?
4. What is the general solution of a non-homogeneous ODE?
5. How is dsolve() used in MATLAB?
6. What happens when the forcing term overlaps with a root of the auxiliary equation?

Solving non-homogeneous ODEs

No.	Differential equation	Complementary function (C.F.)	Particular integral (P.I.)	General solution
1	$y'' - 5y' + 6y = e^{x}$	$C_1 e^{2x} + C_2 e^{3x}$	$\frac{1}{2} e^{x}$	$y = C_1 e^{2x} + C_2 e^{3x} + \frac{1}{2} e^{x}$
2	$y'' - 5y' + 6y = e^{4x}$	$C_1 e^{2x} + C_2 e^{3x}$	$\frac{1}{2} e^{4x}$	$y = C_1 e^{2x} + C_2 e^{3x} + \frac{1}{2} e^{4x}$
3	$y'' - 3y' + 2y = e^{3x}$	$C_1 e^{x} + C_2 e^{2x}$	$\frac{1}{2} e^{3x}$	$y = C_1 e^{x} + C_2 e^{2x} + \frac{1}{2} e^{3x}$
4	$y'' + y = e^{2x}$	$C_1 \cos x + C_2 \sin x$	$\frac{1}{5} e^{2x}$	$y = C_1 \cos x + C_2 \sin x + \frac{1}{5} e^{2x}$
5	$y'' + 4y = \sin x$	$C_1 \cos 2x + C_2 \sin 2x$	$\frac{1}{3} \sin x$	$y = C_1 \cos 2x + C_2 \sin 2x + \frac{1}{3} \sin x$
6	$y'' + 4y = \sin 2x$	$C_1 \cos 2x + C_2 \sin 2x$	$-\frac{x}{4} \cos 2x$	$y = C_1 \cos 2x + C_2 \sin 2x - \frac{x}{4} \cos 2x$
7	$y'' - 2y' + y = e^{x}$	$(C_1 + C_2 x) e^{x}$	$\frac{x^2}{2} e^{x}$	$y = e^{x}(C_1 + C_2 x + \frac{x^2}{2})$
8	$y'' + 2y' + y = e^{-x}$	$(C_1 + C_2 x) e^{-x}$	$\frac{x^2}{2} e^{-x}$	$y = e^{-x}(C_1 + C_2 x + \frac{x^2}{2})$
9	$y'' + 9y = \cos 2x$	$C_1 \cos 3x + C_2 \sin 3x$	$\frac{1}{5} \cos 2x$	$y = C_1 \cos 3x + C_2 \sin 3x + \frac{1}{5} \cos 2x$
10	$y'' + 9y = \cos 3x$	$C_1 \cos 3x + C_2 \sin 3x$	$\frac{x}{6} \sin 3x$	$y = C_1 \cos 3x + C_2 \sin 3x + \frac{x}{6} \sin 3x$
11	$y'' - y = 1$	$C_1 e^{x} + C_2 e^{-x}$	-1	$y = C_1 e^{x} + C_2 e^{-x} - 1$
12	$y'' - 4y = 5$	$C_1 e^{2x} + C_2 e^{-2x}$	$-\frac{5}{4}$	$y = C_1 e^{2x} + C_2 e^{-2x} - \frac{5}{4}$
13	$y'' + y = x$	$C_1 \cos x + C_2 \sin x$	x	$y = C_1 \cos x + C_2 \sin x + x$
14	$y'' - 3y' + 2y = x$	$C_1 e^{x} + C_2 e^{2x}$	$\frac{x}{2} + \frac{3}{4}$	$y = C_1 e^{x} + C_2 e^{2x} + \frac{x}{2} + \frac{3}{4}$
15	$y''' - 6y'' + 11y' - 6y = e^{4x}$	$C_1 e^{x} + C_2 e^{2x} + C_3 e^{3x}$	$\frac{1}{6} e^{4x}$	$y = C_1 e^{x} + C_2 e^{2x} + C_3 e^{3x} + \frac{1}{6} e^{4x}$

* Output :- (1)

Enter the order of the matrix :- 2

Enter the Matrix A : [1 2; 3 4]

Given Matrix A =

1 2

3 4

Eigen Values of the matrix are :

-0.3723

5.3723

* Output :- (2)

Enter the order of the matrix : 2

Enter the matrix A : [1, 2 ; 2 1]

Given matrix A =

1	2
2	1

Eigen values =

-1
3

Corresponding Eigen vectors (columns are eigenvectors) =

-0.7071	0.7071
0.7071	0.7071

Verification of A*V = V*D (2)

A*V =

0.7071	2.1213
-0.7071	2.1213

V*D =

0.7071	2.1213
-0.7071	2.1213

* Exponential Output

* Output :- Equation :- $(e^x - 3x)$

Enter the function f(x) : @(x) exp(x) - 3* x

Enter the lower limit : a : 1

Enter the upper limit b : 2

Iteration	a	b	c	f(c)
1	1.0000	2.0000	1.5000	-0.0183
2	1.5000	2.0000	1.7500	0.5046
3	1.5000	1.7500	1.6250	0.2034
4	1.5000	1.6250	1.5625	0.0832
5	1.5000	1.5625	1.5312	0.0302
6	1.5000	1.5312	1.5156	0.0054
7	1.5000	1.5156	1.5078	-0.0066
8	1.5078	1.5156	1.5117	-0.0006
9	1.5117	1.5156	1.5137	0.0024
10	1.5117	1.5137	1.5127	0.0009
11	1.5117	1.5127	1.5122	0.0001
12	1.5117	1.5122	1.5121	-0.0001
13	1.5120	1.5122	1.5121	0.0000
14	1.5121	1.5121	1.5121	-0.000
15	1.5121	1.5121	1.5121	0.000
16	1.521	1.5121	1.5121	0.000
17	1.521	1.5121	1.5121	0.0000
⋮	⋮	⋮	⋮	⋮

Root = 1.5121

* Trigonometric equation :-

$$\left(\sin x - \frac{x}{2}\right)$$

* Output :-

Enter the function f(x) : @(x) sin(x) - x/2

Enter the lower limit a : 1

Enter the upper limit b : 2

Iteration	a	b	c	f(c)
1	1.0000	2.0000	1.5000	0.2475
2	1.5000	2.0000	1.7500	0.1090
3	1.7500	2.0000	1.8750	0.0166
4	1.8750	2.0000	1.9375	-0.0352
5	1.8750	1.9375	1.9062	-0.0089
6	1.8750	1.9062	1.8906	0.0040
7	1.8906	1.8984	1.8984	-0.0024
8	1.8906	1.8965	1.8945	0.0008
9	1.8945	1.8955	1.8965	-0.0008
10	1.8945	1.8955	1.8955	+ 0.000
11	1.8945	1.8955	1.8950	0.0004
12	1.8950	1.8955	1.8953	0.0002
13	1.8953	1.8955	1.8954	0.0001
14	1.8954	1.8955	1.8954	0.0000
15	1.8954	1.8955	1.8955	0.0000
16	1.8955	1.8955	1.8955	0.0000
17	1.8955	1.8955	1.8955	-0.000
⋮	⋮	⋮	⋮	⋮

Root = 1.8955

* output :- (4)

Enter f(x): @(x) x^3 - 2*x -5

Enter f'(x): @(x) 3*x^2 - 2

Enter initial value : 2

Enter tolerance : 0.0001

Iterations	x	f(x)
1	2.100000	0.061000
2	2.094568	0.000186
3	2.094551	0.000000

Root = 2.094551

Iterations = 3

* Result :- Hence, this is the Newton - Raphson Meth to Solve $x^3 - 2x - 5$.

* Output :- (5)

Iter	x_1	x_2	x_3
1	0.600000	2.272727	−1.100000
2	1.047273	2.227273	−0.992727
3	1.021273	2.277686	−1.086727
4	1.045114	2.266777	−1.076486
5	1.041975	2.269875	−1.082345
6	1.043457	2.269057	−1.081407
7	1.043187	2.269277	−1.081786
8	1.043285	2.269218	−1.081710
9	1.043264	2.269234	−1.081735
10	1.043270	2.26930	−1.081729
11	1.043269	2.269231	−1.081731
12	1.043269	2.269231	−1.08731
13	1.043269	2.269231	−1.08731
14	1.043269	2.269231	−1.081731
15	1.043269	2.269231	−1.081731
16	1.043269	2.269231	−1.081731
17	1.043269	2.269231	−1.081731
18	1.043269	2.269231	−1.081731
19	1.043269	2.269231	−1.081731
20	1.043269	2.269231	−1.081731

Solution :- 1.0433
2.2692
−1.0817



*Output:- (6)

Iter	X1	X2	X3
1	0.600000	2.327273	−0.987273
2	1.030182	2.276628	−1.078374
3	1.043338	2.269542	−1.081713
4	1.043297	2.269235	−1.081736
5	1.043271	2.269230	−1.08173)
6	1.043269	2.269231	−1.081731
7	1.043269	2.269231	−1.081731
8	1.043269	2.269231	−1.081731
9	1.043269	2.269231	−1.08173)
10	1.043269	2.269231	−1.081731

Solution :-

1.0433

2.2692

−1.0817

(2) 1/10/2020

* Output :- (7)

M (x, y) =

2 * x * y + 3

N (x, y) =

x ^ 2 + 4 * y

partial derivative of M with respect to y =

2 * x

Partial derivative of N with respect to x =

2 * x

The differential equation is Exact.

General Solution is :

x * (x * y + 3) + 2 * y ^ 2 = = C

* Output:-(8)

Initial population = 500.00

Growth Constant = 0.02

Time = 10.00 years

Population after 10.00 years = 610.7014

* Output :- (9)

Cooling Const K = 0.03671

Temperature after 20 min = 45.6142 °C

2.]

Temperature after 20 min = 40.0000 °C

3.] Cooling const K = 0.09163

Temperature after 30 min = 33.2000 °C

4.] Temperature after 20 min = 42.8571 °C

Output :- (10)

~~Solution of Non - Homogeneous ODE :~~

~~C1 * exp(x) + C2 * exp(2*x) + exp(3*x)/2~~

Solution of Homogeneous ODE

$C1 * \exp(2*x) + C_2 * \exp(3*x)$

1.] $y'' - 5y' + 2y = 0$

General Solution is :

$C1 * \exp(x) + C2 * \exp(2*x)$

2.] $y'' - 4y' + 4y = 0$

General Solution is :

$(C1 + C2 * x) * \exp(2*x)$

3.] $y'' + 5y' + 6y = 0$

General Solution is :

$C1 * \exp(-2*x) + C2 * \exp(-3*x)$

```
clc;
clear;
close all;
syms y(x)
ode = diff(y,x,2) - 3*diff(y,x) + 2*y
    == exp(3*x);

Sol = dsolve(ode);
disp('General Solution is :');
disp(Sol);
```

Output :- (11)

Solution of Non-homogeneous ODE :

C1*exp(x) + C2*exp(2*x) + exp(3*x)/2

Result :-

This is the ~~given~~ non-homogeneous differential equation was solved using MATLAB

$$y = c_1 e^x + c_2 e^{2x} + \frac{1}{2} e^{3x}$$

Outputs :-

1.] $y'' - 5y' + 6y = e^x$

General Solution is :

C1*exp(2*x) + C2*exp(3*x) + exp(x)/2

2.] $y'' - 3y' + 2y = e^{3x}$ (11)

General Solution is:

$c1 * \exp(x) + c2 * \exp(2*x) + \exp(3*x) + \exp(x)/2$.

3.] $y'' + y = e^{2x}$

General Solution is:

$c1 * \cos(x) + c2 * \sin(x) + \exp(2*x)/5$

4.] $y'' + 4y = \sin(x)$

General Solution is:

$c1 * \cos(2*x) + (2 * \sin(2*x) + \sin(x)/3$

(2)